PROYECTO FINAL · PHP Y MYSQL

Guía de desarrollo paso a paso

GameWorld

Este documento explica **en qué orden construir el proyecto** y **qué hace cada pieza**. No es una lista de código copiable: es una guía de trabajo para entender el porqué de cada paso y poder resolverlo tú mismo en el examen.

La idea central es simple: **cada paso debe quedar probado antes de empezar el siguiente**. Si el catálogo no funciona, el carrito tampoco funcionará; si el carrito no funciona, el pedido fallará. Construir en orden evita perseguir errores en cadena.

PROYECTO
GameWorld

TECNOLOGÍAS
PHP · MySQL · HTML · CSS

ENTREGA
Código, SQL y documento de pruebas

FECHA
Noviembre de 2026

Índice

1. Antes de empezar
2. Organización del código
3. Usuarios
4. Preferencias y tema
5. Catálogo y detalle
6. Carrito
7. Finalizar compra
8. Historial de pedidos
9. Administración
10. Pruebas y documento de entrega
11. Lista de verificación final
12. Anexo: errores típicos

1. Antes de empezar

Tiempo estimado: 10–15 minutos.

1.1 Importar la base de datos

1. Abre phpMyAdmin.
2. Importa `gameworld.sql`.
3. Comprueba que aparecen cuatro tablas: `usuarios`, `videojuegos`, `pedidos` y `detalle_pedido`.

La importación **borra las tablas anteriores** de la base de datos `gameworld`. Si tienes datos que quieres conservar, expórtalos antes.

1.2 Entender el modelo de datos

Antes de escribir una sola consulta, ten claro cómo se relacionan las tablas:

Tabla	Qué guarda	Claves
`usuarios`	Clientes y administradores	`id`, `email` único, `password`, `rol`
`videojuegos`	Catálogo	`id`, `nombre`, `plataforma`, `precio`, `stock`
`pedidos`	Cabecera de cada compra	`id`, `id_usuario`, `fecha`, `total`
`detalle_pedido`	Líneas de cada compra	`id`, `id_pedido`, `id_videojuego`, `cantidad`, `precio_unitario`

Relaciones:

- Un **usuario** tiene muchos **pedidos** (`pedidos.id_usuario`).
- Un **pedido** tiene muchas **líneas** (`detalle_pedido.id_pedido`).
- Un **videojuego** puede aparecer en muchas **líneas** (`detalle_pedido.id_videojuego`).

Esto explica dos reglas del enunciado:

- El historial se filtra por `pedidos.id_usuario`.
- Un videojuego con líneas de pedido **no se puede borrar**, porque rompería el historial.

1.3 Crear el administrador

1. Registra un usuario normal desde la web (cuando el registro ya funcione).
2. Ejecuta en phpMyAdmin:

```
UPDATE usuarios SET rol = 'admin' WHERE email = 'correo@ejemplo.com';
```

Sin un admin no podrás probar la administración. Si todavía no tienes registro, puedes crear el usuario directamente en la base de datos, pero recuerda que la contraseña debe estar generada con `password_hash()`.

1.4 Comprobar el entorno

- MySQL arrancado.
- Servidor PHP arrancado.
- `includes/conexion.php` conecta sin errores.

Comprobación del paso: puedes abrir una página del proyecto sin errores de conexión y ves las cuatro tablas en phpMyAdmin.

---

2. Organización del código

Tiempo estimado: 30–45 minutos.

Este paso no añade funcionalidad visible, pero es el que marca la diferencia entre un proyecto ordenado y un proyecto imposible de corregir. El enunciado lo exige de forma explícita.

2.1 Reparto de responsabilidades

Hay tres capas claras:

Archivo	Responsabilidad	Ejemplos
`includes/funciones_bd.php`	**Todo** el SQL	buscar juegos, insertar usuario, registrar pedido
`includes/funciones.php`	Lógica general reutilizable	validar, redirigir, escapar, CSRF, permisos, totales
Páginas `.php`	Recibir la petición, llamar funciones y pintar HTML	`catalogo.php`, `carrito.php`, etc.

Regla práctica para no equivocarte:

- ¿La función ejecuta una consulta SQL? → `funciones_bd.php`.
- ¿La función sirve para varias páginas pero no habla con la base de datos? → `funciones.php`.
- ¿Es HTML concreto de una sola página? → se queda en esa página.

Nunca escribas un `SELECT`, `INSERT`, `UPDATE` o `DELETE` dentro de una página pública o de administración. El enunciado lo prohíbe y es uno de los errores más penalizados.

2.2 `funciones_bd.php`

Debe contener una función por operación. No una función gigante que lo haga todo.

Agrupa mentalmente las funciones por temas:

- **Catálogo:** listar, buscar con filtros, contar resultados, obtener por ID.
- **Usuarios:** buscar por email, comprobar email, insertar usuario.
- **Administración:** listar todos, insertar, actualizar, borrar, comprobar duplicados, comprobar si un juego tiene pedidos.
- **Pedidos:** registrar pedido completo, listar pedidos de un usuario, listar líneas.

Consejos:

- Usa **sentencias preparadas** siempre que haya datos externos.
- Devuelve arrays o `null`, no imprimas HTML desde aquí.
- Cierra las sentencias (`close()`) cuando termines.
- Para los `LIKE`, construye el comodín en PHP (`%texto%`), no lo pegues en el SQL.

2.3 `funciones.php`

Debe contener helpers que se usan en varias páginas:

- **Redirección:** una función que haga `header('Location: ...')` y `exit`.
- **Escape:** una función que envuelva `htmlspecialchars()` con `ENT_QUOTES` y `UTF-8`.
- **Validación:** texto no vacío, email válido, entero positivo, precio y stock correctos.
- **Sesión y permisos:** `requiereLogin()`, `requiereAdmin()`, saber si hay usuario.
- **CSRF:** generar token, guardarlo en sesión, imprimirlo en el formulario y comprobarlo.
- **Importes:** subtotal de una línea y total del carrito.
- **Tema:** leer la preferencia y decidir la clase CSS.

Estas funciones no deben contener SQL. Si necesitan datos, se los pasa la página o `funciones_bd.php`.

2.4 `header.php` y `footer.php`

Todas las páginas que muestran HTML deben incluirlos con `require_once`.

El header se encarga de:

- Abrir el documento HTML.
- Mostrar la navegación.
- Mostrar enlaces distintos si hay sesión y si el usuario es admin.
- Aplicar el tema claro u oscuro leyendo la cookie.

El footer cierra el documento y carga lo que haga falta al final.

Importante: ocultar un enlace no protege nada. La protección real se hace en el servidor con `requiereAdmin()`.

2.5 Flujo típico de una página

Casi todas las páginas siguen el mismo esqueleto:

1. `require_once` de funciones y conexión.
2. Arrancar sesión si hace falta.
3. Procesar el POST (validar, CSRF, llamar a la función SQL).
4. Redirigir si la operación termina bien (POST-Redirect-GET).
5. Recoger datos para mostrar (GET).
6. Incluir `header.php`.
7. Pintar el HTML escapando los datos.
8. Incluir `footer.php`.

El punto 4 es clave: **redirigir antes de incluir el header**, porque los headers HTTP y las cookies deben enviarse antes de cualquier HTML.

Comprobación del paso: una página cualquiera carga cabecera y pie, y no hay SQL fuera de `funciones_bd.php`.

3. Usuarios

Tiempo estimado: 45–60 minutos.

Por qué ahora: el resto del proyecto depende de saber quién está conectado. El carrito puede existir sin login, pero el pedido, el historial y la administración no.

3.1 Registro

Orden interno:

1. Comprobar que la petición es POST.
2. Comprobar el token CSRF.
3. Recoger y limpiar los campos.
4. Validar: nombre no vacío, email válido, contraseña con longitud mínima.
5. Comprobar si el email ya existe.
6. Hashear la contraseña con `password_hash()`.
7. Insertar el usuario con rol `cliente`.
8. Redirigir al login con un mensaje de éxito.

Detalles que suelen fallar:

- No guardes nunca la contraseña en claro.
- El rol no se recibe del formulario: se asigna en el servidor.
- Si el email está repetido, muestra un error y no insertes.
- Guarda los valores introducidos para no obligar a reescribir todo el formulario.

3.2 Login

Orden interno:

1. POST + CSRF.
2. Validar email y contraseña.
3. Buscar el usuario por email.
4. Si no existe, error genérico.
5. Si existe, comprobar con `password_verify()`.
6. Si coincide, guardar en sesión el ID, el nombre y el rol.
7. Regenerar el ID de sesión para evitar fijación de sesión.
8. Redirigir.

Error genérico: no digas si falló el email o la contraseña. Di “correo o contraseña incorrectos”. Así no revelas qué correos están registrados.

3.3 Logout

- Debe ser POST, porque cambia estado.
- Comprobar CSRF.
- Vaciar los datos de sesión.
- Destruir la sesión.
- Redirigir.

La cookie del tema puede conservarse: es una preferencia, no un dato de autenticación.

3.4 Control de acceso

Dos funciones reutilizables:

- `requiereLogin()`: si no hay usuario en sesión, redirige al login.
- `requiereAdmin()`: además comprueba que el rol sea `admin`.

Se llaman al principio de la página, antes de cualquier salida HTML. No basta con no mostrar el enlace: alguien puede escribir la URL a mano.

3.5 Seguridad de contraseñas

- Al registrar: `password_hash($password, PASSWORD_DEFAULT)`.
- Al entrar: `password_verify($password, $hashGuardado)`.
- Nunca compares contraseñas con `==`.
- Nunca guardes el hash en la sesión.

Comprobación del paso:

- Registro válido → usuario en la base de datos.
- Email repetido → error y no se inserta.
- Contraseña incorrecta → no entra.
- Admin entra y ve Administración.
- Cliente que escribe `/admin` es redirigido.
- Logout deja de tener sesión.

4. Preferencias y tema

Tiempo estimado: 15–20 minutos.

Por qué aquí: solo se toca `header.php` una vez. Si lo dejas para el final también funciona, pero tendrás que volver sobre el header.

4.1 Cómo funciona

1. `preferencias.php` muestra dos opciones: claro y oscuro.
2. Al elegir, se guarda una cookie con el valor durante 7 días.
3. `header.php` lee la cookie y añade la clase CSS correspondiente al `<body>` o al contenedor principal.

4.2 Detalles importantes

- Valida el valor recibido: solo `claro` u `oscuro`. Si llega otra cosa, usa `claro`.
- La cookie debe escribirse **antes** de incluir el header.
- Usa una ruta `/` para que la cookie esté disponible en todas las páginas.
- No guardes el tema en la base de datos: el enunciado pide una cookie.
- El tema debe aplicarse también en login, registro, carrito, admin, etc.

4.3 Por qué una cookie y no una sesión

La sesión desaparece al cerrar el navegador o al hacer logout. La cookie dura 7 días, así que la preferencia sobrevive. Es exactamente lo que pide el enunciado.

Comprobación del paso: eliges oscuro, navegas por varias páginas y sigue oscuro; cierras sesión y el tema se mantiene.

---

5. Catálogo y detalle

Tiempo estimado: 45–60 minutos.

Por qué ahora: necesitas juegos visibles y una ficha de detalle desde la que añadir al carrito.

5.1 Consulta base

El listado necesita:

- Nombre, género, plataforma, precio y stock.
- Solo los campos necesarios, no `SELECT *`.
- Un orden por defecto (por nombre).

5.2 Filtros combinados

Los filtros llegan por GET porque solo leen datos:

- `busqueda`: texto libre.
- `plataforma`: valor exacto.
- `orden`: nombre, precio ascendente o precio descendente.
- `pagina`: número de página.

La consulta se construye por partes:

1. `WHERE 1 = 1` como base.
2. Si hay búsqueda, añade `AND nombre LIKE ?`.
3. Si hay plataforma, añade `AND plataforma = ?`.
4. Añade `ORDER BY` según una lista blanca.
5. Añade `LIMIT ? OFFSET ?`.

Lista blanca para el orden: nunca metas el valor de `orden` directamente en el SQL. Traduce el valor recibido a una de las opciones permitidas. Si no coincide, usa la de defecto.

Parámetros dinámicos: guarda los valores en un array y los tipos en una cadena (`s` para texto, `i` para enteros, `d` para decimales). Pásalos a `bind_param()`.

Cuidado: si no hay ningún filtro, la cadena de tipos puede quedar vacía. En ese caso no llames a `bind_param()`.

5.3 Ordenación

Tres opciones típicas:

- Nombre A–Z.
- Precio de menor a mayor.
- Precio de mayor a menor.

No permitas ordenar por columnas arbitrarias.

5.4 Paginación

1. Cuenta el total de resultados **con los mismos filtros**, sin `LIMIT`.
2. Calcula las páginas: `ceil(total / 5)`.
3. Limita la página recibida al rango válido (mínimo 1, máximo total).
4. Calcula el desplazamiento: `(pagina - 1) * 5`.
5. Pide solo 5 registros.

Errores típicos:

- Contar sin filtros y paginar con filtros.
- No limitar la página y acabar con `OFFSET` negativo.
- Perder los filtros al cambiar de página.

5.5 Conservar los filtros en los enlaces

Los enlaces de paginación y de orden deben incluir búsqueda, plataforma y orden actual. Si no, al pasar a la página 2 se pierde el filtro.

5.6 Detalle

- Recibe un `id` por GET.
- Valida que sea un entero positivo.
- Busca el juego.
- Si no existe, muestra un error o redirige al catálogo.
- Muestra los datos escapados.
- Ofrece el botón de añadir al carrito.

Comprobación del paso:

- Buscar “Mario” y filtrar por Switch devuelve solo los juegos correctos.
- Cambiar el orden por precio funciona.
- La paginación muestra 5 por página y mantiene los filtros.
- Un `id` inexistente no rompe la página.

---

6. Carrito

Tiempo estimado: 45–60 minutos.

Por qué ahora: el catálogo ya permite añadir productos y el checkout necesitará el carrito.

6.1 Estructura en sesión

La forma más sencilla y correcta:

```
$_SESSION['carrito'] = [
    3 => 2,   // id del videojuego => cantidad
    7 => 1,
];
```

Guarda **solo IDs y cantidades**. El nombre, el precio y el stock se consultan siempre en MySQL. Así el usuario no puede manipular el precio desde el navegador.

6.2 Añadir

1. Recibir el ID por POST.
2. Validar que es un entero positivo.
3. Comprobar que el juego existe.
4. Comprobar que hay stock.
5. Si ya está en el carrito, sumar la cantidad.
6. Si la suma supera el stock, rechazar y no modificar.
7. Redirigir al carrito o al catálogo.

6.3 Actualizar cantidades

1. Recibir ID y cantidad por POST.
2. Validar que la cantidad es un entero.
3. Si la cantidad es 0 o menor, eliminar la línea.
4. Si supera el stock, mostrar error y dejar la cantidad anterior.
5. Guardar la nueva cantidad.

6.4 Eliminar

1. Recibir el ID por POST.
2. Quitar esa clave del array de sesión.
3. Redirigir.

6.5 Cálculos

Dos funciones reutilizables:

- `subtotal($precio, $cantidad)`: precio por cantidad, redondeado a 2 decimales.
- `totalCarrito($lineas)`: suma de subtotales.

Las líneas que se pasan a estas funciones deben tener el precio **leído de la base de datos**, no el de la sesión.

6.6 Mostrar el carrito

Para cada ID del carrito:

1. Consulta el juego en MySQL.
2. Si ya no existe, elimínalo del carrito y avisa.
3. Muestra nombre, precio actual, cantidad, subtotal y stock disponible.
4. Ofrece actualizar y eliminar.
5. Muestra el total.

6.7 Refrescar no debe duplicar

Después de añadir o actualizar, redirige. Si no lo haces, al refrescar se repite la operación y las cantidades se suman otra vez.

Comprobación del paso:

- Añadir dos veces el mismo juego acumula.
- Pedir más que el stock se rechaza.
- Actualizar a 0 elimina la línea.
- El total coincide con la suma de subtotales.
- Refrescar no duplica.

---

7. Finalizar compra

Tiempo estimado: 45–60 minutos.

Este es el paso más delicado. Aquí se juntan sesión, base de datos, stock y transacciones.

7.1 Requisitos previos

Antes de tocar la base de datos:

1. El usuario debe estar identificado.
2. La petición debe ser POST.
3. El CSRF debe ser válido.
4. El carrito no puede estar vacío.

Si algo falla, no se crea ningún pedido.

7.2 Qué debe ocurrir, en orden

1. Empezar la transacción (`begin_transaction`).
2. Recorrer el carrito.
3. Para cada juego, consultarlo con `SELECT ... FOR UPDATE`.
4. Comprobar que existe.
5. Comprobar que hay stock suficiente.
6. Guardar el precio real de la base de datos.
7. Calcular el total.
8. Insertar el pedido.
9. Insertar las líneas con el precio unitario.
10. Descontar el stock.
11. Confirmar (`commit`).
12. Vaciar el carrito.
13. Redirigir con mensaje de éxito.

Si cualquier comprobación falla:

1. Deshacer (`rollback`).
2. Mantener el carrito.
3. Mostrar el error.

7.3 Por qué `FOR UPDATE`

`FOR UPDATE` bloquea las filas de los videojuegos mientras dura la transacción. Si dos usuarios compran a la vez el último ejemplar, el segundo espera y, cuando lee el stock, ve el valor ya descontado. Sin el bloqueo, ambos podrían leer el mismo stock y vender de más.

Por eso la comprobación y el descuento deben estar **dentro de la misma transacción**.

7.4 Por qué el precio se lee de la base de datos

El formulario podría enviar un precio falso. El servidor debe ignorarlo y usar `videojuegos.precio`. Lo mismo vale para el total: se calcula en el servidor.

7.5 Vaciar el carrito solo al final

Si vacías el carrito antes del `commit` y algo falla, el usuario pierde su compra. Vacíalo solo cuando la transacción haya terminado correctamente.

7.6 POST-Redirect-GET

Tras un pedido correcto, redirige (por ejemplo, a `mis_pedidos.php`). Así, si el usuario refresca, no se crea un segundo pedido.

7.7 Casos que debes probar

- Sin sesión → redirige al login.
- Carrito vacío → error, sin pedido.
- Stock insuficiente → error, sin pedido y stock intacto.
- Juego borrado mientras estaba en el carrito → error controlado.
- Compra válida → pedido, líneas y stock descontado.
- Refrescar tras la compra → no se duplica.

Comprobación del paso: después de una compra válida, el pedido aparece en la base de datos, las líneas tienen el precio correcto y el stock ha bajado.

---

8. Historial de pedidos

Tiempo estimado: 20–30 minutos.

Por qué ahora: ya existen pedidos reales que mostrar.

8.1 Regla de oro

El historial usa **siempre** el ID del usuario de la sesión. Nunca aceptes un `id_usuario` por la URL.

8.2 Pedidos

1. Comprobar que hay sesión.
2. Consultar los pedidos de ese usuario, ordenados por fecha descendente.
3. Si no hay, mostrar un mensaje.

8.3 Líneas

Para cada pedido:

1. Consultar sus líneas con un `JOIN` a `videojuegos`.
2. Incluir en el `WHERE` el ID del pedido **y** el ID del usuario.
3. Mostrar cantidad, precio unitario y nombre del juego.

Ese segundo filtro es el que impide que un usuario vea líneas de un pedido ajeno aunque manipule el ID.

8.4 Detalle opcional por pedido

Si permites ver un pedido concreto por la URL, vuelve a comprobar que pertenece al usuario de la sesión antes de mostrar nada.

Comprobación del paso: el usuario A ve sus pedidos; si cambia el ID por el de un pedido de B, no ve nada.

---

9. Administración

Tiempo estimado: 60–90 minutos.

Por qué al final: es un CRUD sobre el catálogo que ya conoces. También puedes adelantarlo después de usuarios si quieres tener datos para probar.

9.1 Protección

Todas las páginas de `admin/` deben empezar con `requiereAdmin()`. Sin excepciones.

9.2 Listado

- Mostrar todos los juegos ordenados por nombre.
- Incluir enlaces para editar y eliminar.
- Mostrar el stock y el precio.

9.3 Alta

1. POST + CSRF.
2. Validar nombre, género, plataforma, precio y stock.
3. Comprobar que no existe otro juego con el mismo nombre **y** la misma plataforma.
4. Insertar con sentencia preparada.
5. Redirigir al listado con mensaje.

Validaciones concretas:

- Precio mayor o igual que cero.
- Stock entero mayor o igual que cero.
- Textos no vacíos.
- Plataforma dentro de las permitidas, si el enunciado las limita.

9.4 Edición

1. Recibir el ID por GET para cargar el formulario.
2. Comprobar que el juego existe.
3. Al guardar, validar igual que en el alta.
4. Comprobar duplicados **excluyendo el propio juego**.
5. Actualizar.
6. Redirigir.

El detalle de excluir el propio ID es importante: si no lo haces, editar un juego sin cambiarle el nombre se detectará como duplicado de sí mismo.

9.5 Borrado

1. Recibir el ID por POST (no por GET).
2. Comprobar CSRF.
3. Comprobar que el juego existe.
4. Comprobar si tiene líneas en `detalle_pedido`.
5. Si tiene, mostrar un error explicativo y no borrar.
6. Si no tiene, borrar.
7. Redirigir.

Por qué no se borra con pedidos: el historial quedaría roto. El enunciado pide conservar el producto y avisar.

9.6 Duplicados

La comprobación debe hacerse en el servidor, no solo con JavaScript. La combinación es `nombre + plataforma`. Dos juegos con el mismo nombre en plataformas distintas sí son válidos.

Comprobación del paso:

- Alta válida → aparece en el listado.
- Duplicado → error y no se inserta.
- Edición sin cambiar el nombre → se guarda.
- Borrado de un juego con pedidos → error y sigue existiendo.
- Cliente que entra en `/admin` → redirigido.

---

10. Pruebas y documento de entrega

Tiempo estimado: 30–45 minutos.

10.1 Matriz de pruebas

Prepara una tabla con tres columnas: prueba, resultado esperado y resultado obtenido. Cubre como mínimo:

Caso	Qué compruebas
Registro válido	Se crea el usuario
Email repetido	Se rechaza
Contraseña incorrecta	No entra
Cliente en `/admin`	Se bloquea
Búsqueda + filtro	Devuelve lo correcto
Paginación	Mantiene filtros
Añadir dos veces	Acumula cantidades
Stock insuficiente	Se rechaza
Carrito vacío	No crea pedido
Compra válida	Pedido, líneas y stock
Pedido ajeno	No se puede ver
Alta duplicada	Se rechaza
Borrado con pedidos	Se bloquea
Tema oscuro	Persiste 7 días
Logout	Elimina la sesión

10.2 Capturas

Captura al menos:

- Catálogo con filtros.
- Carrito con líneas y total.
- Pedido realizado o historial.
- Administración.

Guarda las imágenes en una carpeta (`pruebas/`) y referéncialas desde el documento.

10.3 Documento

Estructura sencilla:

1. Entorno (PHP, MySQL, URL local).
2. Tabla de pruebas.
3. Capturas.
4. Notas o incidencias, si las hubo.

No hace falta un documento largo: hace falta que sea claro y verificable.

11. Lista de verificación final

Antes de entregar, repasa esta lista:

- ☐ No hay SQL fuera de `includes/funciones_bd.php`.
- ☐ Todas las páginas usan `require_once` para includes.
- ☐ Todas las páginas HTML incluyen header y footer.
- ☐ Las lecturas usan GET y los cambios de estado usan POST.
- ☐ Todos los formularios que cambian datos tienen CSRF.
- ☐ Todos los datos mostrados pasan por `htmlspecialchars()`.
- ☐ Los permisos se comprueban en el servidor.
- ☐ Las redirecciones y cookies ocurren antes del HTML.
- ☐ Los precios y roles no se confían al formulario.
- ☐ Las escrituras siguen POST-Redirect-GET.
- ☐ El checkout usa transacción, `FOR UPDATE` y rollback.
- ☐ El carrito se vacía solo tras una compra correcta.
- ☐ El historial filtra por el usuario de la sesión.
- ☐ Los duplicados de nombre + plataforma se rechazan.
- ☐ No se borran juegos con líneas de pedido.
- ☐ El tema se guarda en cookie de 7 días.
- ☐ El documento de pruebas existe y tiene capturas.

12. Anexo: errores típicos

Síntoma	Causa probable	Solución
`bind_param(): Argument #1 cannot be empty`	Llamas a `bind_param` sin tipos	Llama solo si hay parámetros
`Cannot modify header information`	HTML antes de `header()`	Redirige antes del header
La paginación pierde filtros	Enlaces sin query string	Añade búsqueda, plataforma y orden
El carrito duplica al refrescar	No rediriges tras el POST	POST-Redirect-GET
Se vende más que el stock	Sin transacción o sin `FOR UPDATE`	Bloquea y comprueba dentro de la transacción
El total no coincide	Precio de la sesión	Lee el precio de la base de datos
Editar detecta duplicado de sí mismo	No excluyes el ID actual	Añade `AND id <> ?`
Cliente entra en admin	Solo ocultas el enlace	`requiereAdmin()` en el servidor
El tema no persiste	Cookie mal configurada	Ruta `/`, 7 días, antes del HTML
Un usuario ve pedidos ajenos	Filtras solo por pedido	Filtra también por usuario

Resumen del orden

1. Base de datos y admin.
2. Includes y convenciones.
3. Usuarios.
4. Tema.
5. Catálogo y detalle.
6. Carrito.
7. Checkout.
8. Historial.
9. Administración.
10. Pruebas y entrega.

Si tienes poco tiempo, el núcleo es **usuarios** → **catálogo** → **carrito** → **checkout**. Administración y preferencias se pueden dejar para después sin romper lo anterior.